Practice 8

Deadline: 2 weeks from now. Should be checked onsite (during labs).

Task 1

In this practice, we'll use executor service as a thread pool to control groups of tasks and increase the efficiency of task execution.

Task 1 - Part 1

Part 1 task lets users input a word, and count the **total occurrences** of this word in all files in a directory tree. Unzip the `src.zip` of JDK 8 and use it as our target directory.

To speed up task execution, we could make a separate task for each file, i.e., count the occurrence of the given word in one file. Since there are many files in a directory, we can submit all these tasks to a thread pool so that they'll execute concurrently. Steps:

1. Create a thread pool using the static factory methods of `Executors`. You may try using different executors (e.g., `CachedThreadPool`,`SingleThreadExecutor`, `FixedThreadPool`) to observe the difference of efficiency.
2. Define a `Callable`, which represents a task that counts the occurrence of the given word in a given file.
3. Traverse the target directory to create a list of such `Callables`.
4. Use executor's `invokeAll` method to run the list of tasks asynchronously, and get back a list of `Future` objects.
5. Sum all the `Future` objects to compute the total occurrence of the given word.

Task 1 - Part 2

In the second part, we search for **the first file that contains the given word**. Again, we use the directory (by unzipping `src.zip`) as our target directory. As long as we found a file (any file) that contains the given word, we consider the task to succeed.

We can use `invokeAny` to parallelize the search tasks, with the following steps:

1. Create a thread pool using the static factory methods of `Executors`.
2. Define a `Callable`, which represents a task that search for the given word in a given file, and returns the file path when search succeeds.
3. Traverse the target directory to create a list of such `Callables`.
4. Use executor's `invokeAny` method to parallel the search, and get back the path of the first file that succeeds the search.

We have to be more careful about formulating the tasks in step 2. Since the `invokeAny` method terminates as soon as any task returns, we cannot have the search tasks return a `boolean` to indicate success or failure, because we don't want to stop searching when a search task failed. Instead, a failing search task (i.e., the file doesn't contain the given word) should throws a `NoSuchElementException`.

Also, when one task has succeeded, the others are canceled. Therefore, we monitor the interrupted status. If the underlying thread is interrupted, the search task prints a message before terminating, so that you can see that the cancellation is effective. You may use the following code for this purpose:

```
if (Thread.currentThread().isInterrupted())
{
    System.out.println("Search in " + path + " canceled.");
    return null;
}
```

We provide an incomplete code `ExecutorPractice.java` on BB. You may use it as a starting point and complete the TODOs. Sample output:

```
Enter keyword: socket
Occurrences of socket: 926
Found the first file that contains socket:
src\java\rmi\server\UnicastRemoteObject.java
Search in src\javax\xml\bind\ValidationException.java canceled.
Search in src\com\sun\imageio\spi\OutputStreamImageOutputStreamSpi.java canceled.
Search in src\javax\imageio\spi\ImageReaderSpi.java canceled.
Search in src\org\w3c\dom\html\HTMLTitleElement.java canceled.......
```

Task 2

In this task, we'll implement a simple Pokémon query service in which multiple clients could simultaneously query a server to ask for basic information about a given pokémon.

Server

The server should keep listening for clients. Once the server establishs a connection with a client, it could get the name of a pokémon sent by the client. Then, the server should make a REST request to PokeAPI to get the basic information about this pokémon.

PokeAPI (https://pokeapi.co/) is a free RESTful service for querying Pokémon data. For instance, we could query all data w.r.t a specific pokémon using its Pokemon endpoint:

`GET https://pokeapi.co/api/v2/pokemon/{id or name}/`

If the client sent "pikachu", the server should:

- Query the data for "pikachu" using the Pokemon endpoint.
- Get the JSON response.
- Retrieve pikachu's height, weight, and a list of abilities by analyzing the JSON data (you may use any JSON APIs or libraries for parsing).
- Send the information back to the client.

Client

After connecting to the server, the client could keep typing names of pokémon, and print out the information about this pokémon sent from the server. If the client sent "QUIT", the connection will be closed.

Demo

You should start the server, and start at least two clients at the same time.

Server

```
Waiting for clients to connect...

Client connected.
Info of pikachu sent

Client connected.
Info of arbok sent
Info of psyduck sent
Client quits.
Client quits.
```

Client 1

```
Enter the pokemon name: pikachu
Name: pikachu
Height: 4
Weight: 60
Abilities: [static, lightning-rod]

Enter the pokemon name: psyduck
Name: psyduck
Height: 8
Weight: 196
Abilities: [damp, cloud-nine, swift-swim]

Enter the pokemon name: QUIT
```

Client 2

```
Enter the pokemon name: arbok
Name: arbok
Height: 35
Weight: 650
Abilities: [intimidate, shed-skin, unnerve]

Enter the pokemon name: QUIT
```